

gstack+: A Three-Tier LLM Orchestration Framework for Cost-Efficient AI Development

Dave Zhang

Technical Report · May 2026

Abstract

We present gstack+, a three-tier AI task orchestration framework that reduces LLM API costs by 46%–98% while maintaining or improving output quality. The framework routes development tasks to the most appropriate model tier — Tier-A (Architect), Tier-Mid (Reviewer), or Tier-Exec (Executor) — via a five-dimension scoring system (Judgment, Context, Risk, Verifiability, Creativity). Across eight independent experiment series comprising 110+ tasks spanning four technical domains, gstack+ achieves 100% routing accuracy in three independent validations, and the S1 prompt strategy produces quality scores of 15.0/15 versus Opus’s 12.7/15 at 86.7% lower cost. Sensitivity analysis reveals that the Risk (R) and Judgment (J) dimensions each affect routing decisions in 32–33% of boundary cases, motivating a conservative routing rule. These results collectively demonstrate that accurate task routing, not model capability, is the primary determinant of AI development workflow efficiency.

摘要 (Chinese Abstract)

本文提出 gstack+，一個三層 AI 任務編排框架，通過五維度評分系統（判斷強度 J、上下文寬度 C、風險權重 R、可驗證性 V、創意密度 Cr）將開發任務路由至最合適的模型層（Tier-A 架構師、Tier-Mid 審查員、Tier-Exec 執行者），在不損失品質的前提下削減 46%–98% 的 LLM API 成本。八個獨立實驗系列、110+ 個任務、4 個技術領域的驗證表明：路由準確率在三次獨立驗證中均達 100%；最優 Prompt 策略（S1）在 Tier-Mid 任務中以 86.7% 更低成本超越 Opus（15.0 vs 12.7 分）；敏感性分析揭示 R 和 J 維度各影響 32–33% 邊界任務的路由決策，支持保守路由原則。實驗結果一致表明，準確的任務路由——而非模型能力本身——是 AI 輔助開發 workflow 效率的主要決定因素。

Keywords: LLM orchestration, task routing, cost optimization, AI-assisted development, prompt engineering, multi-model systems

1 Introduction

The rapid adoption of large language models (LLMs) in software development workflows has introduced a new cost dimension: model API expenses can account for a significant fraction of engineering budgets at scale. Despite this, current practice overwhelmingly relies on a single, typically top-tier model for all tasks — a strategy we term **model selection flattening**.

This creates two classes of systematic waste. First, **over-routing**: sending mechanical tasks such as cross-repository renaming or documentation formatting to the most expensive models, paying a 20–100× cost premium for zero quality benefit. Second, **under-routing**: sending high-judgment tasks such as security architecture design or cross-cutting refactoring to cheap models, introducing quality risk and decision errors whose remediation costs far exceed any savings.

The central insight of gstack+ is that **most development tasks do not require the most capable model**, but identifying which tasks do requires a principled classification framework. We observe that real-world development tasks roughly follow a 50/35/15 distribution across three complexity tiers — a distribution that, combined with a 30× cost spread between cheapest and most expensive tier models, offers substantial savings under correct routing.

Our contributions are:

1. A **five-dimension scoring framework** (J/C/R/V/Cr) with deterministic routing rules that achieves 100% accuracy across 70 tasks in three independent validations.
2. A **three-tier model architecture** (Tier-A/Mid/Exec) with formalized role boundaries and handoff protocols.
3. **Empirical evidence** from 8 experiment series showing 46%–98% cost reduction without quality degradation.
4. **Sensitivity analysis** quantifying which scoring dimensions most influence routing decisions, motivating practical safeguards.

2 Related Work

2.1 LLM Cost Optimization

Prior work on LLM cost reduction focuses primarily on model compression (quantization, distillation) and inference optimization (speculative decoding, KV-cache sharing). These approaches reduce per-token cost but do not address the task-model mismatch. Frugal-GPT [CITATION] introduces a similar routing concept but focuses on API-level cost negotiation rather than task-complexity-aware dispatch. Our approach is complementary: we operate at the workflow orchestration layer.

2.2 Multi-Agent Systems

AutoGen [CITATION], CrewAI, and LangGraph provide multi-agent frameworks but focus on agent **roles and communication patterns** rather than **cost-aware task routing**. They do not provide a principled scoring framework for deciding which agent capability level a task requires. gstack+ fills this gap with a quantitative 5-dimension classifier.

2.3 Prompt Engineering for Quality

The finding that carefully engineered prompts can make smaller models outperform larger ones has been demonstrated in chain-of-thought [CITATION] and role-based prompting work. Our Series 3 results confirm and extend this: Sonnet with the S1 (role + deep-think) strategy achieves 15.0/15 versus Opus at 12.7/15 on Tier-Mid tasks. This supports our thesis that routing + prompt strategy jointly determine output quality, not model size alone.

3 System Design

3.1 Design Motivation

The core hypothesis of gstack+ is: **routing correctness is more important than model capability for the majority of development tasks.**

This hypothesis rests on two observations. First, task complexity follows a skewed distribution: approximately 50% of real development tasks involve mechanical execution (renaming, reformatting, documentation), 35% involve moderate judgment (refactoring with design decisions, code review), and only 15% require deep architectural reasoning. Second, the cost differential between model tiers is approximately 30×, meaning that correctly routing the bottom 50% of tasks to cheaper models can reduce total spend by 46% even without touching the top tier.

The implication is that a framework should invest in **classification accuracy** rather than always defaulting to the most capable model.

3.2 Five-Dimension Scoring Framework

Each task is evaluated on five dimensions, each scored 1–5:

Dim.	Name	What it measures	Scale
J	Judgment	Human-level decision-making required	1–5
C	Context	Codebase knowledge breadth needed	1–5
R	Risk	Cost of getting it wrong	1–5
V	Verifiability	Can success be automatically verified?	1–5
Cr	Creativity	Novel design required (vs. templated)	1–5

Table 1: Five-dimension scoring framework for task classification

The five dimensions were selected to cover three capability axes: **cognitive load** (J + Cr), **knowledge scope** (C), and **risk posture** (R + V). This decomposition ensures that tasks are not over-simplified (a single “difficulty” score misses important distinctions between a risky-but-simple task and a complex-but-safe one).

3.3 Three-Tier Routing Rules

Routing decisions are deterministic given the five scores:

Tier-A : $J \geq 4$ OR $R \geq 4$ OR $Cr \geq 4$
Tier-Exec: $J \leq 2$ AND $C \leq 2$ AND $V \geq 4$ AND $R \leq 2$
Tier-Mid : All remaining cases (conservative default)

The asymmetry between Tier-A and Tier-Exec rules is intentional. Tier-A triggers on **any single high-stakes signal** (OR logic) because the downside of under-routing a risky task is severe. Tier-Exec requires **all conditions** to hold (AND logic) because mechanical execution benefits from strict verification requirements.

3.4 Conservative Routing Principle

Sensitivity analysis (Series 6) reveals that J and R each influence routing in 32–33% of boundary cases with ± 1 score perturbations. This motivates a **conservative routing rule**: when $J = 3$ or $R = 3$ (boundary values), route to Tier-A rather than Tier-Mid.

The economic justification: the cost of under-routing a high-judgment task (remediation, rework, potential production issues) typically exceeds the cost of over-routing by 3–5 $\times$. The conservative rule adds at most 20–50% extra cost in boundary cases while avoiding catastrophic failures.

3.5 Tier Architecture and Handoff Protocol

Each tier has clearly defined responsibilities and communication boundaries:

Tier-A (Architect) decomposes user requirements into tasks, applies the 5-dimension scoring, makes routing decisions, and reviews escalated failures. It does not write implementation code.

Tier-Mid (Reviewer) validates Tier-Exec outputs against defined success criteria, applies evidence-based acceptance/rejection, and produces structured feedback for Tier-A. It can fix minor issues but cannot override architectural decisions.

Tier-Exec (Executor) implements tasks according to the handoff specification (scope-locked), produces structured evidence reports, and escalates blockers. It cannot make architectural judgments.

Handoffs between tiers use structured templates (plan-to-exec.md, exec-to-check.md, check-to-plan.md) that enforce explicit scope locks, verifiable success criteria, and evidence requirements. This protocol eliminates ambiguous completions (“it looks done”) that are a major source of rework in unstructured AI workflows.

4 Experimental Setup

4.1 Experiment Series Overview

We conducted eight independent experiment series between 2026-05-04 and 2026-05-06, each targeting a specific validation question:

Series	Research Question	Tasks
S1	Baseline cost vs. quality comparison (3 tiers)	3
S2	Routing accuracy, stability, and cost efficiency at scale	39
S3	Prompt strategy impact; real git-history routing validation	24
S4	Cross-domain generalization (4 technical domains)	20
S5	Multi-model quality matrix (Haiku/Sonnet/Opus $\times$ 3 tiers)	9
S6	Scoring dimension sensitivity (± 1 perturbation analysis)	10
S7	Routing error asymmetry and over/under-routing cost analysis	8
S8	Quality matrix refinement and conservative routing validation	6

Table 2: Experiment series overview

4.2 Evaluation Protocol

Quality evaluation uses **LLM-as-Judge** blind scoring (judge model: claude-opus-4-7) on five dimensions (technical correctness, completeness, clarity, risk awareness, practical value), each scored 0–3 for a maximum of 15. Blind scoring eliminates confirmation bias — the judge does not know which model produced each output.

Cost measurements use actual API token consumption recorded from IDE billing data, converted to USD at published list prices. All experiments use identical task descriptions and starting git states across modes to ensure fair comparison.

5 Results and Analysis

5.1 Series 1–2: Cost Reduction and Quality Parity

The most direct result is from the Series 1 cost baseline:

Task	Routing	All-Opus	Routed	Saved
Cross-repo rename	Tier-Exec $\rightarrow$ Qwen	\$0.01173	\$0.00014	–99%
React Query refactor	Tier-Mid $\rightarrow$ Sonnet	\$0.07849	\$0.01191	–85%
SSO+MFA design	Tier-A $\rightarrow$ Opus	\$0.07885	\$0.07885	—
Total		\$0.1691	\$0.0909	–46%

Table 3: Series 1 cost comparison across three representative tasks

Critically, routing did not degrade quality: the Tier-Mid task (React Query refactor) routed to Sonnet **outperformed** Opus (5/5 vs 4/5 in human evaluation). Series 2 Exp-2C confirmed this with blind LLM-as-Judge scoring: routing mode and All-Opus mode both scored 14.1/15.

5.2 Series 3: Prompt Strategy Dominates Model Size

Series 3 reveals that prompt strategy is the primary quality determinant for Tier-Mid tasks:

Strategy	Description	Score (/15)	Cost/task
S0 (baseline)	No prompt engineering	13.7	\$0.006
S2	Structured output format	13.3	\$0.007
S3 (CoT+role)	Chain-of-thought + role	15.0	\$0.007
S1 (best)	Role + deep-think	15.0	\$0.006
Opus baseline	No prompt eng., Opus	12.7	\$0.045

Table 4: Prompt strategy quality comparison on Tier-Mid tasks (Series 3)

The S1 strategy achieves 15.0/15 with Sonnet at \$0.006/task — a 15.7% quality **improvement** over Opus at \$0.045/task, for an 86.7% cost reduction.

5.3 Series 4: Cross-Domain Generalization

Series 4 validates the framework across Frontend, Backend, Data Engineering, and DevOps domains (20 tasks, 5 per domain). An AI agent (Qwen Code) applied the scoring guide independently, without domain-specific training or calibration.

Result: 20/20 routing accuracy, with zero mean score deviation across all five dimensions in all four domains. This confirms that the 5-dimension framework is sufficiently well-defined for cross-domain deployment without adaptation.

5.4 Series 5–6: Multi-Model Comparison and Sensitivity

Series 5 reveals a nuanced picture of model capability: Haiku 4.5 scores 14/15 on Tier-Exec tasks (only 1 point below Opus) and **outperforms** Sonnet and Opus on Tier-Mid tasks (14 vs 13). Opus’s only systematic advantage is in the Risk Awareness evaluation dimension on Tier-A tasks (2.7 vs Haiku’s 2.0).

Series 6 sensitivity analysis shows that the Risk (R=33%) and Judgment (J=32%) dimensions are most likely to change routing decisions under ± 1 perturbation. Verifiability (V=11%) and Creativity (Cr=13%) are most stable. This motivates targeting scoring effort toward R and J in ambiguous cases.

6 Discussion

6.1 Limitations

Sample size: The core dataset covers 110+ tasks across 8 series. While results are consistent, statistical significance claims require larger validation sets. The 100% routing accuracy result, while robust across three independent validations, should be interpreted as “no failures observed” rather than “provably perfect.”

Controlled environment: All experiments use known, pre-labeled tasks. Real-world deployment introduces novel task types that may not fit cleanly into the three-tier taxonomy.

Single-user evaluation: LLM-as-Judge scoring, while blind, uses a single judge model. Cross-judge validation would strengthen quality claims.

6.2 Future Work

Three extensions are most promising: (1) online adaptation of routing thresholds based on observed failure rates, (2) cost-aware routing that incorporates dynamic pricing into tier assignment, and (3) multi-judge evaluation protocols to improve quality measurement robustness.

7 Conclusion

gstack+ demonstrates that systematic task routing, rather than capability maximization, is the key lever for efficient AI-assisted development. The five-dimension framework achieves 100% routing accuracy across 70 tasks in three independent validations, the three-tier architecture reduces costs by 46%–98% without quality degradation, and sensitivity analysis provides actionable guidance for scoring edge cases. The core insight — that 80% of development tasks do not require the most expensive model — is simple, but its systematic application requires a principled framework. gstack+ provides that framework with quantitative validation.

8 References

- [1] Brown, T. et al. (2020). Language Models are Few-Shot Learners. NeurIPS.
- [2] Chen, L. et al. (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176.
- [3] Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS.
- [4] Wu, Q. et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.
- [5] Anthropic. (2024). Claude 3 Model Card. Technical Report.
- [6] Zhang, D. (2026). gstack+ Experiment Series 1–8. Technical Report, available at github.com/zhewenzhang/gstack-plus.